

Exercise Sheet 9

Lab Sessions: July 2/3, 2026

In this exercise sheet, we work with JSON, a hierarchical data format that is central to storing and exchanging (research) data.

1 Preparation

For this exercise, we provide files in the [RepEng FIMGit Repository](#) (directory `LabSession9/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

2 JSON and JSON Schema

Consider data about births: for each month of a year, we record the number of *male*, *female*, and *total* births. The file `births.json` stores these observations as a JSON array, with one object per month.

2.1 Docker Environment

The directory `LabSession9/` contains a `Dockerfile` that installs the command-line tools used in the first part of this sheet, together with the data files `births.json` and `births_schema.json`. Build the image and start a temporary interactive container:

Terminal Session

bash

```
user@host$ cd LabSession9
user@host$ docker build -t lab9-json .
user@host$ docker run -it lab9-json
```

2.2 Pretty-Printing JSON

JSON data found “in the wild” is frequently stored as a single line, without indentation or line breaks. This is space-efficient, but hard to read. Inspect the raw file first:

Terminal Session

bash

```
user@container$ cat births.json
[{"year":2018,"month":"January","male":4500,"female":4300,"total":8800},{
  "year":2018,"month":"February","male":4200,"female":3990,"total":8190},{
  "year":2018,"month":"March","male":4750,"female":4510,"total":9260},{
  "year":2019,"month":"July","male":5100,"female":4860,"total":9960},{
  "year":2019,"month":"August","male":5230,"female":4970,"total":10200},{
  "year":2019,"month":"September","male":4880,"female":4660,"total":9540}]
```

We use `jq`, a lightweight command-line JSON processor, to convert between the representations: *pretty-printed* and *compacted*.

Pretty-printing reformats the document with consistent indentation. The identity filter `.` returns the input unchanged, but `jq` pretty-prints its output by default:

Terminal Session

bash

```
user@container$ jq . births.json > births_pretty.json
user@container$ cat births_pretty.json
```

Compacting is the inverse operation: it strips all insignificant whitespace, producing the compact single-line form again. To compact, pass the `-c` (`--compact-output`) flag to `jq`:

Terminal Session

bash

```
user@container$ jq -c . births_pretty.json
```

Both directions preserve the underlying data; only the formatting changes.

When is each representation useful?

Please continue on the next page

2.3 Querying JSON with jq

Beyond reformatting, jq can extract and transform data. This is handy when we only want to inspect part of a large file.

1. Print only the `month` field of every record:

```
Terminal Session bash  
user@container$ jq '.[].month' births.json
```

2. Count the number of records in the file:

```
Terminal Session bash  
user@container$ jq 'length' births.json
```

3. Select all records of the year 2019 and keep only the `month` and `total`:

```
Terminal Session bash  
user@container$ jq '.[] | select(.year == 2019) | {month, total}' \  
    births.json
```

Using these examples as a template, write jq queries of your own that:

- (a) print only the `year` field of every record;
- (b) print the `total` for August.

Note: In the exam, you are not expected to know the jq syntax by heart. However, you might need to apply a documented feature (similar to this task).

2.4 Comparing JSON Documents

Two JSON documents can look very different – different indentation, a different order of the keys within an object – and yet represent exactly the same data.

The files `births_reordered.json` and `births.json` are equivalent. They have the structure and hold the same data but their formatting differs: `births_reordered.json` uses line breaks and indentation, and within each object the keys appear in another order.

1. First, compare the two files as they are using `diff`:

```
Terminal Session bash  
user@container$ diff births.json births_reordered.json
```

Why is this comparison of little help to decide whether the two files contain the same data?

2. Devise a way to compare the two files that ignores differences in *formatting* and *key order*.

Hint: Use `jq -S` to sort keys within JSON objects.

2.5 Writing JSON Schema

JSON Schema is a standard to describe the structure a JSON document must follow. A JSON document is *valid* for the schema if it does not violate the structure described by the schema.

Write a schema in JSON Schema that specifies the structure you observed in `births.json`. Create a basic schema for which `birth.json` is valid: enforce data types and required/optional properties. Store the schema in a file named `births_schema.json`.

2.6 Validating JSON against a Schema

JSON Schema validators check the validity of a JSON document for a schema.

- (a) Validate `births.json` against `births_schema.json` using `check-jsonschema`¹:

Terminal Session

bash

```
user@container$ check-jsonschema --schemafile births_schema.json \
    births.json
```

1. Run the command above. What does the tool report?
 2. If the result is not “ok”, refine your schema until the document is valid for it.
- (b) The repository also contains `births_broken_1.json` and `births_broken_2.json`, in which some records violate the usual structure.
1. Validate `births_broken_1.json` against the schema.
 2. If the result is “ok”, refine your schema until the document is invalid for it, while making sure that `births.json` remains valid for the schema.
 3. If the result is not “ok”, how does the tool react, and what does the error message tell you?
 4. Repeat the steps above for `births_broken_2.json`

Please continue on the next page

¹<https://check-jsonschema.readthedocs.io/>

2.7 Reproducible Validation with Containers: Bowtie

We just validated our data with *one* validator. But the JSON Schema specification is complex, and different validators do not always agree: they may support different subsets of the specification, contain bugs, or change their behavior between versions. For a result to be reproducible, it is not enough to say “the data is valid” – we must also specify *which validator, in which exact version*, produced that verdict.

The framework Bowtie addresses this issue. It is a *meta-validator*: rather than validating itself, it runs JSON Schema validators and collects their results. Each validator runs inside its own Docker container. The container image, identified by a name and tag, freezes a validator with all necessary dependencies at a specific version. Re-running the same image later yields the same validator, and thus a reproducible result.

Bowtie orchestrates these containers itself, so we run it on the host, not inside the `lab9-json` container. Install it by following the instructions in the [online documentation](#)². Note that the latest version of Bowtie requires at least Python 3.13³; with older Python versions, some of the commands below might not be available.

After installing Bowtie, perform the following steps:

1. List the supported implementations. Each entry corresponds to a container image:

Terminal Session

bash

```
user@host$ bowtie filter-implementations
```

2. Validate `births.json` against `births_schema.json` using the Python implementation `python-jsonschema`, and summarise the result. The first run pulls the validator’s container image:

Terminal Session

bash

```
user@host$ bowtie validate -i python-jsonschema \
    births_schema.json births.json | bowtie summary
```

Hint: Ensure that both `births.json` and `births_schema.json` are in your current working directory.

3. Now run the *same* schema and instance against a *second* implementation written in another language, for example `js-ajv`, and compare:

Terminal Session

bash

```
user@host$ bowtie validate -i python-jsonschema -i js-ajv \
    births_schema.json births.json | bowtie summary
```

Explain, in terms of reproducibility, why running the validator inside a pinned container image is preferable to installing a validator directly on your machine.

²<https://docs.bowtie.report/en/stable/>

³In `uv`, you can specify the Python version: `uv tool install bowtie-json-schema --python 3.13`

3 JSON and JSON Schema (Antwort-Wahl-Verfahren)

In this task, each question has exactly one correct answer. Mark the correct answer.

(a) Consider the following JSON Schema (draft 2020-12):

```
schema.json
{
  "type": "object",
  "additionalProperties": false,
  "required": ["id", "score"],
  "properties": {
    "id": { "type": "integer", "minimum": 1 },
    "name": { "type": "string" },
    "score": { "type": "number", "minimum": 0, "maximum": 100 }
  }
}
```

How many of the following instances are valid for this schema?

- (1) {"id":1,"name":"Ada","score":90} (2) {"id":0,"name":"Bob","score":50}
(3) {"id":3,"name":"Cy","score":80,"x":1} (4) {"id":4,"score":100}

☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

(b) Using the *same* schema as in the previous question, how many of the following instances are valid for the schema?

- (1) {"id":5,"score":95} (2) {"id":6,"name":"Finn","score":"95"}
(3) {"id":7,"name":"Gus","score":60.5} (4) {"id":8,"name":"Hank","score":0}

☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

Please continue on the next page

(c) Consider the following JSON Schema (draft 2020-12), which uses `oneOf`:

schema.json

```
{
  "type": "object",
  "required": ["hero", "level"],
  "properties": {
    "hero": { "type": "string" },
    "level": { "type": "integer", "minimum": 1, "maximum": 100 },
    "gadget": { "type": "string" },
    "mutation": { "type": "string" }
  },
  "oneOf": [
    { "required": ["gadget"] },
    { "required": ["mutation"] }
  ]
}
```

How many of the following instances are valid for this schema?

- (1) {"hero": "Volt", "level": 42, "gadget": "taser"}
- (2) {"hero": "Blaze", "level": 9, "mutation": "fire"}
- (3) {"hero": "Rex", "level": 50, "gadget": "claw", "mutation": "speed"}
- (4) {"hero": "Zed", "level": 0, "gadget": "net"}

☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4

(d) Using the *same* schema as in the previous question, how many of the following instances are valid for the schema?

- (1) {"hero": "Nova", "level": 100, "mutation": "gravity"}
- (2) {"hero": "Gale", "level": 30, "gadget": "boots", "city": "Metro"}
- (3) {"hero": "Ping", "level": 25}
- (4) {"hero": "Ace", "level": 77, "mutation": "cloak"}

☐ 0 ☐ 1 ☐ 2 ☐ 3 ☐ 4